Instructor Guide — Build an Agentic RAG System from Scratch

> **Target audience:** Students who know Python and basic web development but have never built a production RAG pipeline. After following this guide they will have reproduced the Al-Khwarizmi AI Assistant end-to-end.

Table of Contents

1. What Are We Building?
2. Why These Tools?
3. Architecture Overview
4. Environment Setup
5. Phase 1 — Document Ingestion Pipeline
6. Phase 2 — The LangGraph Agent
7. Phase 3 — The FastAPI Backend
8. Phase 4 — The Angular Frontend
9. Phase 5 — Governance and Safety
10. Phase 6 — Observability
11. Phase 7 — Evaluation
12. Phase 8 — Docker and Deployment
13. Full File-by-File Reference
14. Key Concepts Deep Dive
15. Common Mistakes to Avoid
16. Grading Rubric

---

1. What Are We Building?

We are building a **multi-system bilingual (Arabic + English) RAG chatbot** for the Al-Khwarizmi survey platform. The system can answer questions about four sub-products:

System	Description
`designer`	Survey form builder
`runtime`	Survey execution engine
`callcenter`	Call center operator console
`admin`	Platform administration

Core capabilities

- Understands Arabic (including Gulf dialect), English, and mixed questions.
- Routes each question to the correct sub-system's document collection.
- Returns grounded answers with numbered citations (`[1]` , `[2]` , …).
- Streams tokens to the browser in real-time over **Server-Sent Events (SSE)**.
- Attaches relevant UI screenshots to answers via semantic image matching.
- Remembers multi-turn conversation threads (Redis or JSON file).
- Guards against prompt injection, over-long inputs, and rate abuse.

---

2. Why These Tools?

This section explains every major technology decision so you can defend it in a code review.

2.1 LangGraph — the core reason to use a graph

Most tutorials chain LangChain components with `|` (the LCEL pipe). That works for a single linear flow. This project needs:

1. A **conditional branch**: after retrieval, go to `answer` if the docs are relevant, else `fallback` .
2. A single **authoritative list** (`PRE_ANSWER_PIPELINE`) used by both the compiled graph **and** the SSE streaming path — no duplication.
3. Easy future extension: add a `clarification` node, a `rerank` node, or loop back for query expansion without rewriting everything.

LangGraph models the pipeline as a **directed graph** over a shared `TypedDict` state. Each node is a plain Python function that receives the full state and returns a partial update dict.

LangGraph merges the partial updates. This is the right mental model:

```
START
  │
  ▼
language_detect    (heuristic, no LLM)
  │
  ▼
rewrite_and_route  (1 LLM call → JSON)
  │
  ▼
retrieval          (Chroma + BM25 + RRF, no LLM)
  │
  ├── relevance == "relevant" ──► answer   (1 LLM call)  ──► END
  │
  └── relevance != "relevant" ──► fallback (static text)  ──► END
```

Cost: 2 LLM calls per question in the happy path. The project previously had 5 (language, rewrite, route, relevance grader, answer). Every removed LLM call reduces latency and API cost.

2.2 Anthropic Claude

- Best-in-class instruction following for bilingual structured output (the rewrite+route node returns JSON and Claude rarely hallucinates the schema).
- Long context window (200K tokens) is not needed here but useful for large document answers.
- `langchain-anthropic` provides the same interface as all other LangChain LLMs so swapping to GPT-4 requires changing one environment variable.

2.3 Chroma (vector database)

- Runs **in-process** as a persistent directory — no separate database server to install for development.
- Supports `similarity_search_with_score` which returns the raw L2 distance used for the relevance gate.
- One folder per system (`vector_stores/designer/` , `vector_stores/admin/` , …) keeps collections isolated and fast to rebuild individually.
- `langchain-chroma` wraps it with LangChain's `VectorStore` interface.

2.4 `intfloat/multilingual-e5-large` (embeddings)

- One of the best open-weight multilingual embedding models, strong on Arabic.
- Runs locally (no API key, no cost per token) with `sentence-transformers` .
- Produces **L2-normalised** vectors, so dot-product equals cosine similarity — used directly in the flow-image semantic matching.
- `normalize_embeddings=True` is mandatory: without it L2 distances are not comparable across queries.

2.5 Hybrid Retrieval (Dense + BM25 + RRF)

Pure embedding search fails on exact Arabic terms, UI labels, and product-specific codes. BM25 (Okapi) scores lexical term overlap. Reciprocal Rank Fusion (RRF) merges both rankings without fragile score normalisation:

```
RRF_score(doc) = 1/(k + rank_dense) + 1/(k + rank_bm25)
```

`k=60` is the standard default from the original RRF paper. Raise it to reduce the impact of rank position; lower it to amplify top-rank advantage.

2.6 FastAPI + SSE

- FastAPI is async-native, type-checked via Pydantic, and auto-generates OpenAPI docs at `/docs` .
- `sse-starlette` wraps an async generator into a proper `text/event-stream` response.
- The client only needs a regular POST request and an SSE reader — no WebSocket handshake or reconnect protocol.

2.7 Angular 21 (standalone components)

- `fetch()` -based SSE is more flexible than `EventSource` (allows POST body with `question` and `thread_id`).
- Standalone components (no NgModules) reduce boilerplate.
- `RxJS Observable` wraps the fetch-based SSE for easy subscription management and cleanup.

2.8 Redis (optional thread memory)

- Thread history is stored as Redis lists (`rag:thread:<id>:turns`).
- Falls back to JSON files under `memory/threads/` when `REDIS_URL` is not set — zero infrastructure for local dev.
- `MEMORY_MAX_TURNS=10` keeps the prompt from growing unboundedly.

3. Architecture Overview

```
┌──────────────────────────────────────────────────────────────────────┐
│                          OFFLINE / SETUP                             │
│                                                                      │
│  docs/                                                               │
│  ├── designer/designer.md  ──┐                                       │
│  ├── admin/admin.md          ├── python -m ingestion                 │
│  ├── callcenter/callcenter.md│      ▼                                 │
│  └── runtime/runtime.md    ──┘  Chroma stores                       │
│                                 vector_stores/                       │
│                                 ├── designer/                        │
│                                 ├── admin/                           │
│                                 ├── callcenter/                      │
│                                 └── runtime/                         │
└──────────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────────┐
│                        ONLINE / REQUEST                              │
│                                                                      │
│  Browser (Angular)                                                   │
│      │ POST /chat  (SSE)                                             │
│      ▼                                                               │
│  FastAPI  api/main.py                                                │
│      │ governance check + thread memory load                         │
│      ▼                                                               │
│  LangGraph  agent/graph.py                                           │
│      │                                                               │
│      ├─► language_detect_node   (agent/nodes.py, text_ar.py)         │
│      ├─► rewrite_and_route_node (Claude API → JSON)                  │
│      ├─► retrieval_node         (Chroma + BM25 + RRF)                │
│      │                                                               │
│      ├── relevant ──► answer_node    (Claude API → tokens)           │
│      └── irrelevant ► fallback_node  (static bilingual text)         │
│                                                                      │
│      SSE token stream ──► browser                                    │
└──────────────────────────────────────────────────────────────────────┘
```

4. Environment Setup

4.1 Prerequisites

Tool	Version	Notes
Python	3.11+	Use pyenv or conda
Node.js	20+	For Angular frontend
Git	any	
Docker (optional)	24+	For Redis or full stack

4.2 Python virtual environment

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt
```

4.3 Environment variables

Copy `.env.example` to `.env` and fill in at minimum:

```
ANTHROPIC_API_KEY=sk-ant-...          # from console.anthropic.com
EMBEDDING_MODEL=intfloat/multilingual-e5-large
LLM_MODEL=claude-sonnet-4-20250514
VECTOR_STORE_PATH=./vector_stores
LOG_PATH=./logs
MEMORY_PATH=./memory
ALLOWED_ORIGINS=http://localhost:4200
```

All other variables have sensible defaults (see `.env.example` for the full list with explanations).

4.4 Download the embedding model

The first time `get_embeddings()` is called it downloads `multilingual-e5-large` from Hugging Face (~1.2 GB). Pre-download it:

```
from sentence_transformers import SentenceTransformer
SentenceTransformer("intfloat/multilingual-e5-large")
```

After this, set `local_files_only=True` in `agent/nodes.py` to prevent accidental re-downloads in production.

---

5. Phase 1 — Document Ingestion Pipeline

Goal: Turn Markdown/PDF/DOCX manuals into searchable Chroma vector stores.

5.1 Folder contract

```
docs/
├── designer/
│   └── designer.md        # full product manual
├── admin/
│   └── admin.md
├── callcenter/
│   └── callcenter.md
└── runtime/
    └── runtime.md
```

The system name in `SYSTEMS` must match the folder name. This is the single source of truth.

5.2 How ingestion works (`ingestion/`)

`ingestion/config.py` — constants loaded from `.env`:

- `DOCS_PATH` — where to find source documents (default `./docs`)
- `INGEST_CHUNK_SIZE=1100` — characters per chunk
- `INGEST_CHUNK_OVERLAP=150` — overlap so no sentence is split across chunks

- `INGEST_CLEAN_STORE=true` — delete `vector_stores/<system>/` before rebuilding (idempotent)

`ingestion/documents.py` — loads files and enriches metadata:

1. Uses `UnstructuredMarkdownLoader`, `PyPDFLoader`, `Docx2txtLoader` via `langchain-community`.
2. Calls `enrich_with_section_context()` to prepend the heading breadcrumb to each chunk's metadata (`section_path`, `system_label`). This means every chunk "knows" which heading it came from, improving answer precision when the LLM cites `[3]`.
3. Splits with `RecursiveCharacterTextSplitter(chunk_size=1100, chunk_overlap=150)`.

`ingestion/chroma_ingest.py` — writes to Chroma:

1. Calls `ingest_system(system, embeddings)` for each system.
2. Writes a `.metadata.json` file next to the Chroma directory recording `embedding_model`, `ingest_time`, `chunks` count. The API reads this at startup to warn when the wrong model was used.

`ingestion/ingest.py` — CLI entry:

```
# Ingest all systems
python -m ingestion

# Ingest only designer
python -m ingestion designer
```

5.3 Chunk metadata fields

Every chunk stored in Chroma has:

Field	Value	Purpose
`system`	`"designer"` etc	System filter during retrieval
`source_file`	absolute path	Citation display
`source_rel`	relative path	Shorter citation display
`language`	`"ar"` / `"en"`	Language filter during retrieval
`section_path`	`"Settings > Skip Logic"`	Answer heading context
`system_label`	`"Survey Designer"`	Human-readable label
`image_refs`	JSON list of filenames	Screenshots linked to this chunk
`ingest_source`	`"folder"` / `"monolith"`	Provenance

5.4 Optional monolith

When `KNOWLEDGE_MONOLITH=./shared.md`, one shared document is ingested into **all four** collections. This is useful for platform-wide policies or shared glossaries.

---

6. Phase 2 — The LangGraph Agent

Files: `agent/state.py`, `agent/graph.py`, `agent/nodes.py`, `agent/retrieval.py`, `agent/prompts.py`, `agent/text_ar.py`

6.1 The State (`agent/state.py`)

Every node receives the full state and returns a partial dict. LangGraph merges the partial update.

```
class AgentState(TypedDict):
    question: str                           # raw user input
    language: Literal["ar", "en", "mixed"] # set by language_detect_node
    rewritten_question: str                 # set by
rewrite_and_route_node
    system: Optional[str]                   # "designer" | "admin" | … |
"none"
    retrieved_chunks: list[str]             # plaintext chunks
    retrieved_source_refs: list[dict]       # citation metadata
    retrieval_best_distance: float | None   # top-1 L2 distance from
Chroma
    image_urls: list[str]                   # /images/... paths
    relevance: Literal["relevant","irrelevant","unknown"]
    answer: str                             # final answer text
    thread_id: str
    conversation_history: list[dict]        # prior turns
    request_id: str                         # for tracing
```

Student task: Add a new field `confidence_score: float` and have `answer_node` populate it.

6.2 The Graph (`agent/graph.py`)

```
def build_graph():
    graph = StateGraph(AgentState)

    # Add all pre-answer nodes from the single source of truth
    for name, fn in PRE_ANSWER_PIPELINE:
        graph.add_node(name, fn)

    graph.add_node("answer", answer_node)
    graph.add_node("fallback", fallback_node)

    # Wire linear edges
    graph.add_edge(START, "language_detect")
    graph.add_edge("language_detect", "rewrite_and_route")
    graph.add_edge("rewrite_and_route", "retrieval")

    # Conditional branch
    graph.add_conditional_edges(
        "retrieval",
        route_by_relevance,           # returns "answer" or "fallback"
        {"answer": "answer", "fallback": "fallback"}
    )

    graph.add_edge("answer", END)
    graph.add_edge("fallback", END)

    return graph.compile()
```

`PRE_ANSWER_PIPELINE` is defined in `nodes.py`:

```
PRE_ANSWER_PIPELINE: tuple[tuple[str, NodeFn], ...] = (
    ("language_detect",   language_detect_node),
    ("rewrite_and_route", rewrite_and_route_node),
    ("retrieval",         retrieval_node),
)
```

Using this tuple as the single source of truth means the SSE path in `api/main.py` iterates the same list, keeping graph and streaming in perfect sync without any code duplication.

6.3 Node 1 — Language Detection (`language_detect_node`)

No LLM. Uses character script ratios from `agent/text_ar.py`:

- Count Arabic Unicode block characters vs Latin characters.
- If >70% Arabic → `"ar"`, if >70% Latin → `"en"`, else `"mixed"`.

Why not use an LLM? Speed. This runs in microseconds, costs nothing, and is deterministic.

6.4 Node 2 — Rewrite and Route (`rewrite_and_route_node`)

One LLM call. Prompt instructs Claude to return **exactly this JSON**:

```
{
  "rewritten_question": "How do I add skip logic to a question in the survey designer?",
  "system": "designer"
}
```

Why combine rewrite and route into one call?

- Rewriting a question for search and routing to a sub-system require the **same context** (the question + conversation history).
- Separating them would waste a second LLM call with identical input.

The prompt (`REWRITE_AND_ROUTE_JSON_PROMPT` in `agent/prompts.py`) contains:

1. The list of valid systems with descriptions.
2. Instructions to resolve pronouns using conversation history ("it", "that feature").
3. Instructions to return `"system": "none"` for off-topic questions.
4. JSON-only output instructions.

`_parse_rewrite_route_json()` is defensively written — it strips markdown fences, finds the outermost `{}` braces, and tolerates extra keys.

6.5 Node 3 — Retrieval (`retrieval_node`)

No LLM. Steps:

1. If `system == "none"`, return empty chunks and `relevance = "irrelevant"`.
2. Optionally normalise Arabic (remove diacritics, bridge Gulf dialect to MSA).
3. Call `hybrid_retrieve()` from `agent/retrieval.py`.
4. Decide relevance: chunk count ⩾ `RETRIEVAL_MIN_RELEVANT_CHUNKS` → relevant (skips L2 gate).
5. Optionally match the rewritten question against `screens.json` flow embeddings to find relevant UI screenshots.

6.6 Hybrid Retrieval (`agent/retrieval.py`)

```
Dense pool  = Chroma.similarity_search_with_score(query, k=32)
                          |
                          ▼
BM25 over dense pool only  (not over the entire corpus)
                          |
                          ▼
RRF fusion: score(doc) = 1/(60+rank_dense) + 1/(60+rank_bm25)
                          |
                          ▼
[Optional] CrossEncoder rerank  top-24 → top-8
                          |
                          ▼
Deduplicate (same file + same 200-char prefix)
                          |
                          ▼
Return top-K documents + best_l2_distance
```

Why BM25 only over the dense pool? Indexing the entire corpus in BM25 at query time would be slow. The dense pool (32 docs) is already semantically relevant; BM25 just reorders it based on exact token matching. This is the classic "BM25 over candidate pool" pattern from the ColBERT paper.

6.7 Arabic Text Processing (`agent/text_ar.py`)

Function	Purpose
`detect_language_for_rag()`	Script ratio → `"ar"` / `"en"` / `"mixed"`

`normalize_arabic_question()`	Remove diacritics, normalize alef/ya variants
`apply_gulf_vocab_bridge()`	Map Gulf dialect words to MSA equivalents
`camel_get_lemmas()`	CAMeL Tools lemmatization (optional, degrades gracefully)
`is_arabic_stopword()`	Remove high-frequency Arabic function words before BM25

Gulf dialect examples: `بلدر` → `designer`, `كول سنتر` → `callcenter`, `أين` → `وين`.

6.8 Node 4 — Answer (`answer_node`)

One LLM call. The answer prompt (`ANSWER_PROMPT`) provides:

- The language detected (`ar` or `en`) — Claude responds in the same language.
- Numbered context chunks with heading breadcrumbs.
- Instructions to cite sources with `[1]`, `[2]`, etc.
- Conversation history for coherence.

For SSE, `astream_answer_tokens()` uses `llm.astream()` to yield token deltas.

6.9 Node 5 — Fallback (`fallback_node`)

No LLM. Checks three cases:

- **Greeting** (`مرحبا`, `hello`, `hi`, ...) → warm welcome message.
- **Platform overview** (`شو الخوارزمي`, `what is al-khawarzmi`) → product summary.
- **Off-topic / no results** → polite redirect message.

All messages are bilingual — Arabic or English selected by `state["language"]`.

6.10 Image Selection (`_select_images_for_question`)

`docs/screens.json` maps UI flows to screenshot filenames:

```
{
  "flows": [
    {
      "id": "skip_logic",
      "system": "designer",
      "desc": { "en": "Add skip logic to a question", "ar": "إضافة منطق تخطي للسؤال" },
      "steps": ["step1.png", "step2.png"]
    }
  ],
  "images": { "step1.png": { "ar": "step1_ar.png", "en": "step1_en.png" } }
}
```

At startup, `_ensure_flow_embeddings()` batches all flow descriptions through the embedding model. At query time, the rewritten question is embedded and dot-product compared to all flow vectors. If `similarity ≥ 0.65`, the matching flow's images are included in the response.

---

7. Phase 3 — The FastAPI Backend

File: `api/main.py`

7.1 Lifespan

```
@asynccontextmanager
async def _lifespan(app: FastAPI):
    setup_observability()   # configure logging
    prom_init()             # register Prometheus counters
    get_compiled_graph()    # compile LangGraph (validates nodes/edges)
    get_embeddings()        # load multilingual-e5-large into RAM
    logger.info("API startup complete — ready to serve requests")
    yield
    logger.info("API shutdown complete")
```

Pre-warming the embedding model on startup means the first user request is fast — no cold start delay.

7.2 Middleware

Every request (including health checks) goes through `request_observability_middleware`:

- Generates `X-Request-ID` (UUID4, or passthrough from client).
- Sets a `contextvars.ContextVar` so all log lines in the same request carry the same `rid`.
- Emits a JSON access log with method, path, status code, and duration.

7.3 Routes

Method	Path	Auth	Purpose
`GET`	`/health`	None	Liveness probe (always 200 if the process is running)
`GET`	`/health/ready`	Optional	Readiness probe (503 when Chroma dirs missing)
`GET`	`/metrics`	None	Prometheus scrape
`POST`	`/chat/sync`	Optional API key	Blocking full-pipeline response (tests, simple clients)
`POST`	`/chat`	Optional API key	SSE streaming response (the Angular UI uses this)
`GET`	`/images/*`	None	Static file serving for UI screenshots

7.4 SSE Protocol

The `/chat` endpoint streams events in this order:

```
event: node_start
data: {"node": "language_detect"}

event: node_start
data: {"node": "rewrite_and_route"}

event: node_start
data: {"node": "retrieval"}

event: node_start
data: {"node": "answer"}

event: token
data: The survey

event: token
data:  designer allows

... (one event per token delta)

event: done
data: {"thread_id": "abc123", "images": ["/images/step1.png"]}
```

If governance blocks the request:

```
event: blocked
data: {"code": "policy_violation", "message": "...", "thread_id":
"abc123"}

event: done
data: {"thread_id": "abc123", "blocked": true}
```

7.5 Thread Memory

```
thread_id = resolve_thread_id(request.thread_id)   # generates UUID if
None
history = load_conversation(thread_id)              # Redis list or JSON
file
# … run the graph …
append_turn(thread_id, question, answer)            # persist for next
turn
```

Threads expire after `MEMORY_MAX_TURNS=10` turns (oldest dropped) to bound prompt size.

---

8. Phase 4 — The Angular Frontend

Directory: `RAG_FE/`

8.1 Project structure

```
RAG_FE/src/app/
├── app.ts            # shell component, router outlet
├── chat/
│   ├── sse-chat.component.ts   # main UI: sidebar, message list, input
│   ├── thread.store.ts          # thread list state
│   └── render.ts                # markdown-like text → safe HTML
└── rag/
    └── rag-chat.service.ts      # SSE client, event parsing
```

8.2 SSE Client (`rag-chat.service.ts`)

Because `EventSource` only supports GET, the service uses `fetch()` with POST and manually reads the stream:

```
const response = await fetch(`${baseUrl}/chat?api_key=${key}`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ question, thread_id: threadId }),
});

const reader = response.body!.getReader();
// parse lines: "event: token\ndata: hello\n\n"
```

Event types handled:

Event	Action
`node_start`	Show pipeline progress indicator
`token`	Append to current message buffer
`done`	Finalise message, store `thread_id`, display images
`blocked`	Show policy violation message
`error`	Show error toast

8.3 Thread Store

`ThreadStore` keeps the list of conversation threads in memory (and optionally `localStorage`). Each thread is identified by the `thread_id` returned in the `done` event. The user can click a previous thread to continue the conversation.

8.4 RTL support

The chat service detects Arabic text using a simple regex and sets `dir="rtl"` on the message element. The Angular component uses `[dir]` binding so mixed conversations render each message in the correct direction.

8.5 Environment configuration

```
// src/environments/environment.ts
export const environment = {
  apiBaseUrl: 'http://localhost:8000'
};
```

In production, override by setting `localStorage.RAG_API_BASE_URL` in the browser console (useful for demos without a rebuild).

---

9. Phase 5 — Governance and Safety

File: `agent/governance.py`

9.1 What governance does (in order)

1. **Empty / too-long question** → `422` (client error, not 403 — important distinction).
2. **Prompt injection heuristics** — patterns like `ignore previous instructions`, `system:`, `<|im_start|>` → `403`.
3. **Blocklist** — optional `GOVERNANCE_BLOCKLIST_FILE` with one phrase per line → `403`.
4. **Rate limit** — optional `GOVERNANCE_RATE_LIMIT_PER_MINUTE=60` per client IP → `429`. Uses Redis counters in multi-worker production; in-memory dict for single process.

9.2 Audit log

When `GOVERNANCE_AUDIT_LOG=./logs/governance_audit.jsonl` is set, blocked attempts are logged as JSONL. Only the first 40 characters of the question plus a SHA256 hash are stored — enough for pattern analysis without retaining user data.

9.3 Sanitised question

`evaluate_question()` returns a `GovernanceOutcome` with `sanitized_question` — the question after removing control characters and normalising whitespace. The sanitised version (not the raw input) is passed to the graph.

---

10. Phase 6 — Observability

File: `agent/observability.py`

10.1 Structured logging

Set `OBSERVABILITY_JSON_STDOUT=true` for log stacks (Datadog, CloudWatch). Default format is human-readable with `[rid=… tid=…]` prefixes.

Every log line in the same request carries `request_id` and `thread_id` via `contextvars.ContextVar`. This allows filtering an entire conversation in a log aggregator with one query.

10.2 Query log (`queries_YYYY-MM-DD.jsonl`)

Every completed question appends a JSON line to `logs/queries_YYYY-MM-DD.jsonl`:

```
{
  "timestamp": "2026-05-10T14:32:01",
  "request_id": "abc",
  "thread_id": "xyz",
  "question": "كيف أضيف منطق التخطي؟",
  "language": "ar",
  "system": "designer",
  "rewritten_question": "How do I add skip logic to a question?",
  "chunks_count": 7,
  "citations_count": 5,
  "relevance": "relevant",
  "retrieval_best_distance": 0.34,
  "fallback": false,
  "coverage_gap": false,
  "answer_preview": "لإضافة منطق التخطي…"
}
```

`coverage_gap=true` when the answer contains phrases like "I don't have enough information". Use these logs to identify missing documentation topics.

10.3 Prometheus

`GET /metrics` exposes Prometheus exposition format. Register custom counters in `prom_init()`. Scrape interval: 15s recommended.

---

11. Phase 7 — Evaluation

Files: `eval/golden.json`, `eval/run_eval.py`

11.1 Golden set structure

`eval/golden.json` contains 52 test cases:

```
{
  "version": 2,
  "cases": [
    {
      "id": "designer_skip_logic_ar",
      "question": "كيف أضيف منطق التخطي في المصمم؟",
      "expect_system": "designer",
      "expect_fallback": false,
      "keywords_any": ["منطق التخطي", "skip logic", "شرط"],
      "min_citations": 1
    }
  ]
}
```

Field	Meaning
`expect_system`	Expected routing target
`expect_fallback`	`true` for off-topic / greeting questions
`keywords_any`	At least one of these must appear in the answer
`keywords_all`	All of these must appear in the answer
`min_citations`	Minimum `[n]` citation count in the answer

11.2 Running evaluation

```
python eval/run_eval.py
# Writes eval/reports/latest.json
```

The report shows per-case pass/fail and aggregate scores. Run this after every prompt change.

11.3 Unit tests

```
pytest tests/test_queries.py -v
```

Tests that do **not** require the LLM:

- Governance input validation.
- Language detection accuracy.
- Fallback/greeting detection.

Tests that require LLM (skipped by default if `ANTHROPIC_API_KEY` is not set):

- Full graph invocation for a sample question.

---

12. Phase 8 — Docker and Deployment

12.1 Dockerfile (multi-stage)

```
# Stage 1: builder
FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# Stage 2: runtime
FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY agent/ agent/
COPY api/ api/
COPY ingestion/ ingestion/
COPY eval/ eval/
# Non-root user for security
RUN useradd -m rag
USER rag
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000",
     "--workers", "4", "--loop", "uvloop", "--http", "httptools"]
```

12.2 docker-compose.yml

```
services:
  rag-api:
    build: .
    ports: ["8000:8000"]
    env_file: .env
    environment:
      REDIS_URL: redis://redis:6379/0
    volumes:
      - ./vector_stores:/app/vector_stores   # pre-built Chroma dirs
      - ./memory:/app/memory
      - ./logs:/app/logs
      - ./docs:/app/docs
    depends_on:
      redis:
        condition: service_healthy

  redis:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
    volumes: [redis_data:/data]

volumes:
  redis_data:
```

12.3 Deployment checklist

- [ ] Run ingestion on the host before `docker compose up` (Chroma dirs are volume-mounted).
- [ ] Set `RAG_REQUIRE_VECTOR_STORES=true` in production to fail fast if dirs are missing.
- [ ] Set `RAG_API_KEY` to a random secret and configure the Angular `environment.prod.ts`.
- [ ] Set `ALLOWED_ORIGINS` to the production Angular URL only.
- [ ] Set `OBSERVABILITY_JSON_STDOUT=true` for log aggregation.
- [ ] Configure a reverse proxy (nginx/Caddy) for TLS termination.

13. Full File-by-File Reference

Backend

File	Lines	Responsibility
`api/main.py`	~559	FastAPI app, lifespan, SSE, auth, thread memory wiring
`agent/graph.py`	~73	LangGraph StateGraph compilation
`agent/state.py`	~31	`AgentState` TypedDict
`agent/nodes.py`	~722	All node functions, `PRE_ANSWER_PIPELINE`, query logging
`agent/retrieval.py`	~324	Hybrid dense+BM25+RRF, cross-encoder, deduplication
`agent/prompts.py`	~N/A	All prompt strings (rewrite, answer, fallback messages)
`agent/text_ar.py`	~N/A	Arabic normalisation, Gulf dialect bridge, CAMeL lemmas
`agent/governance.py`	~N/A	Input validation, injection heuristics, rate limit, audit
`agent/thread_memory.py`	~N/A	Redis or JSON file conversation history
`agent/observability.py`	~N/A	Logging setup, Prometheus, contextvars, access log
`agent/vector_health.py`	~N/A	Chroma directory readiness checks
`agent/env_utils.py`	~N/A	`EMBEDDING_MODEL`, `env_bool()` helper
`agent/paths.py`	~N/A	`PROJECT_ROOT`, `vector_store_root()`, screens.json loader
`ingestion/config.py`	~N/A	Ingestion constants from `.env`
`ingestion/documents.py`	~N/A	Document loaders, section enrichment, splitter
`ingestion/chroma_ingest.py`	~N/A	Per-system Chroma build, monolith upsert
`ingestion/ingest.py`	~44	CLI entry point
`eval/run_eval.py`	~N/A	Offline golden-set evaluation
`eval/golden.json`	~463	52 test cases
`tests/test_queries.py`	~N/A	Pytest unit + optional integration tests

Frontend

File	Responsibility
`RAG_FE/src/app/rag/rag-chat.service.ts`	SSE fetch client, event parsing
`RAG_FE/src/app/chat/sse-chat.component.ts`	Main chat UI
`RAG_FE/src/app/chat/thread.store.ts`	Thread list state
`RAG_FE/src/app/chat/render.ts`	Safe markdown-like renderer
`RAG_FE/src/environments/environment*.ts`	`apiBaseUrl` per environment

---

14. Key Concepts Deep Dive

14.1 Why not use LangChain LCEL chains?

LCEL (`prompt | llm | parser`) is great for simple linear pipelines. It falls short when you need:

- **Conditional routing**: LCEL does not have native `if/else` branching.
- **Shared mutable state**: LCEL passes output of one step as input to the next. There is no shared dict that all steps can read from.
- **SSE node events**: You need to know when each node **starts** to emit `node_start` events. LCEL has no node lifecycle hooks.

LangGraph solves all three. The tradeoff is slightly more setup code.

14.2 Why TypedDict for state (not a Pydantic model)?

LangGraph's `StateGraph` is designed around TypedDict. Pydantic models work too but add overhead and the `.model_dump()` / `.model_validate()` roundtrip at every node boundary. TypedDict is zero-overhead — it is just a type hint dict at runtime.

14.3 Relevance gate without an LLM

Earlier versions used a separate Claude call to grade `relevant` / `irrelevant`. That was:

- Slow (extra round trip).
- Expensive.
- Unreliable (the grader LLM hallucinated "relevant" for off-topic questions too often).

The current approach uses two heuristics:

1. **Chunk count** — if hybrid retrieval returned ⩾ 3 chunks, there is enough signal to attempt an answer.
2. **L2 distance** — if the top-1 dense score is too far from the query (configured by `RETRIEVAL_RELEVANCE_MAX_L2`), mark irrelevant.

Both can be tuned in `.env` without code changes.

14.4 Normalised embeddings and dot-product

When embeddings are L2-normalised (`||v|| = 1`), the **dot product equals cosine similarity**. This property is used in `_select_images_for_question`:

```
sim = sum(a * b for a, b in zip(q_vec, fvec))
```

This is valid only because `get_embeddings()` is created with `encode_kwargs={"normalize_embeddings": True}`.

14.5 Reciprocal Rank Fusion (RRF) — the math

```
RRF(doc, k) = Σ_rank_lists  1 / (k + rank(doc))
```

`k=60` smooths out the benefit of being ranked #1 vs #2 vs #3. Lower `k` makes the top-ranked docs more dominant; higher `k` gives more weight to consistent mid-list presence.

Example with `k=60`:

- Doc A: rank 1 in dense, rank 3 in BM25 → `1/61 + 1/63 = 0.0321`
- Doc B: rank 5 in dense, rank 1 in BM25 → `1/65 + 1/61 = 0.0318`
- Doc C: rank 2 in dense, rank 2 in BM25 → `1/62 + 1/62 = 0.0323`

Doc C wins because it is consistently near the top in both lists.

14.6 SSE vs WebSocket

	SSE	WebSocket
Direction	Server → Client only	Bidirectional
Protocol	HTTP/1.1 (text stream)	Upgraded TCP
Browser support	All modern browsers	All modern browsers
Firewall/proxy	Rarely blocked	Sometimes blocked
POST body	Only via `fetch()` trick	Native
Auto-reconnect	Built-in with `EventSource`	Manual

For this use case (one-way token streaming per request), SSE is simpler and more reliable.

---

15. Common Mistakes to Avoid

Ingestion

❌ **Ingesting without `normalize_embeddings=True`** — chunks and queries will be in incompatible spaces.

❌ **Not cleaning the store before re-ingesting** — Chroma will append duplicates. Set `INGEST_CLEAN_STORE=true`.

❌ **Chunk size too large (>2000 chars)** — the LLM context window fills up; citation precision drops.

❌ **Chunk overlap too small (<100 chars)** — sentences at chunk boundaries are cut and retrieval misses them.

Agent

❌ **Passing `state["question"]` (raw) instead of `state["rewritten_question"]` to the answer node** — for follow-up questions like `"explain more"` the raw question makes no sense to the LLM.

❌ **Not using `local_files_only=True` in production** — the embedding model may attempt to download on every deploy, causing delays or failures in air-gapped environments.

❌ **Mutable default arguments** — never `def node(state: AgentState, history=[])` in Python. Use `None` as default and assign inside.

API

❌ **Not pre-warming embeddings at startup** — the first request after a cold start takes 5–10 extra seconds while the model loads.

❌ **Putting secrets in CORS `allow_origins=["*"]`** — if you set `allow_credentials=True`, wildcard origins are blocked by browsers for security reasons. Always list explicit origins.

❌ **Not handling `OSError` on thread memory writes** — file system full or read-only mount should not crash the API; log and continue.

Frontend

❌ **Using `EventSource` for the SSE endpoint** — `EventSource` only supports GET. The chat endpoint requires POST to send `question` and `thread_id` in the body.

❌ **Not handling the `done` event** — the `thread_id` for multi-turn is only available in the `done` payload; don't rely on guessing it.

---

16. Grading Rubric

Milestone	Weight	Criteria
M1: Ingestion	15%	Chroma stores build cleanly; `.metadata.json` present; chunks have correct metadata fields
M2: Language detection	5%	Correctly classifies 10 sample questions (Arabic / English / mixed)
M3: Rewrite + Route	20%	Routes correct system ⩾85% on golden set; handles Gulf dialect; parses JSON reliably
M4: Hybrid retrieval	20%	RRF fusion implemented; BM25 tokenisation is Arabic-aware; deduplication works
M5: Answer + Fallback	15%	Grounded answers cite `[1]`, `[2]`; fallback covers greetings and off-topic; bilingual
M6: SSE streaming	10%	Browser receives tokens in order; `node_start` events appear; `done` carries `thread_id`
M7: Governance	5%	Long input → 422; injection attempt → 403; rate limit → 429
M8: Evaluation	10%	All 52 golden cases run; ⩾80% pass rate on routing + fallback
Bonus: Docker	+10%	`docker compose up` starts the full stack; `/health/ready` returns 200

This guide was written to be the only document a student needs to reproduce the Al-Khwarizmi AI Assistant from scratch. Read each phase in order, implement it, verify it passes the relevant milestone, then move to the next.